

Java Backend Architecture

Interview Series

Chapter 17.9

GKE & EKS – Managed Kubernetes

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 17.9 — GKE & EKS: Managed Kubernetes

Overview

GKE (Google Kubernetes Engine) and EKS (Amazon Elastic Kubernetes Service) are the two dominant managed Kubernetes platforms. Both manage the control plane (etcd, API server, scheduler). They differ in node management models, identity federation, and cloud service integration.

GKE: Standard vs Autopilot

GKE Standard vs Autopilot: Responsibility Boundary

GKE Standard		GKE Autopilot	
Control plane	Google		Google
Node provisioning	YOU		Google
Node OS patching	YOU		Google
Node autoscaling	YOU configure CA		Google (auto)
Resource limits	Optional		Required (enforced)
Billing unit	Per node (VM)		Per pod (vCPU+mem)

Autopilot: No SSH to nodes | DaemonSets not allowed | Workload Identity enforced

- **GKE Standard:** you manage node pools (VM type, size, count), OS patching, Cluster Autoscaler config
- **GKE Autopilot:** Google manages nodes — pay per pod vCPU+memory; no node management
- **Node Pools:** groups of nodes with same VM type/OS; separate pools for spot, GPU, high-memory
- **Release Channels:** Rapid/Regular/Stable — controls k8s version auto-upgrade cadence
- **GKE Autopilot enforces:** resource requests required, no privileged containers, no DaemonSets

```
# GKE Autopilot cluster creation gcloud container clusters create-auto my-cluster \ --region
us-central1 \ --release-channel regular # GKE Standard with node pools gcloud container
clusters create my-cluster \ --zone us-central1-a \ --machine-type n2-standard-4 \ --num-nodes
3 \ --enable-autoscaling --min-nodes=1 --max-nodes=10 \ --workload-pool=my-project.svc.id.goog
# Workload Identity # Add spot/preemptible node pool gcloud container node-pools create
spot-pool \ --cluster my-cluster \ --spot \ --machine-type n2-standard-2 \ --enable-autoscaling
--min-nodes=0 --max-nodes=20
```

GKE Workload Identity

- **Problem:** pods need GCP IAM permissions (GCS, BigQuery, Pub/Sub) without key files
- **Solution:** k8s ServiceAccount annotated with GCP Service Account email
- GKE binds k8s SA to GCP SA — pod token exchanged for GCP credentials via OIDC
- No key file management, no key rotation, per-pod IAM isolation

```
# 1. Create GCP Service Account gcloud iam service-accounts create app-gsa \
--project=my-project # 2. Grant GCP SA permissions gcloud projects add-iam-policy-binding
my-project \ --member="serviceAccount:app-gsa@my-project.iam.gserviceaccount.com" \
--role="roles/storage.objectViewer" # 3. Allow k8s SA to impersonate GCP SA gcloud iam
service-accounts add-iam-policy-binding app-gsa@my-project.iam.gserviceaccount.com \
```

```
--member="serviceAccount:my-project.svc.id.goog[app-ns/app-sa]" \
--role="roles/iam.workloadIdentityUser" # 4. Annotate k8s SA kubectl annotate serviceaccount
app-sa \ --namespace app-ns \
iam.gke.io/gcp-service-account=app-gsa@my-project.iam.gserviceaccount.com
```

GKE Additional Features

- **Binary Authorization:** admission control policy requiring container images to be signed (Cosign/Sigstore)
- **Cloud Armor:** WAF + DDoS protection attached to GCP Load Balancer (Ingress-level)
- **Config Connector:** manage GCP resources (CloudSQL, Pub/Sub) as k8s CRDs
- **GKE Dataplane V2:** eBPF-based networking (Cilium) replacing kube-proxy
- **Confidential GKE Nodes:** AMD SEV for memory encryption (compliance)

Amazon EKS

- **Managed control plane:** AWS runs HA etcd (3 AZs), API server; you pay per cluster/hour
- **Managed node groups:** AWS manages node lifecycle (EC2 ASG, node updates, draining)
- **Self-managed nodes:** you manage EC2 instances, AMIs, autoscaling — full control
- **Fargate profiles:** serverless pods — no node management, pay per pod vCPU+memory
- **EKS Blueprints:** Terraform/CDK modules for opinionated EKS cluster setup
- **EKS Anywhere:** run EKS on-premise (VMware, bare metal)

EKS IRSA — IAM Roles for Service Accounts

EKS IRSA: OIDC Token Exchange Flow



SA annotation: `eks.amazonaws.com/role-arn: arn:aws:iam::123:role/app-role`

IAM Trust Policy: `condition sts:RoleSessionName = SA name`

No long-lived credentials | Auto-rotated | Pod-level isolation

SDK auto-detects projected token at `/var/run/secrets/eks.amazonaws.com/serviceaccount/token`

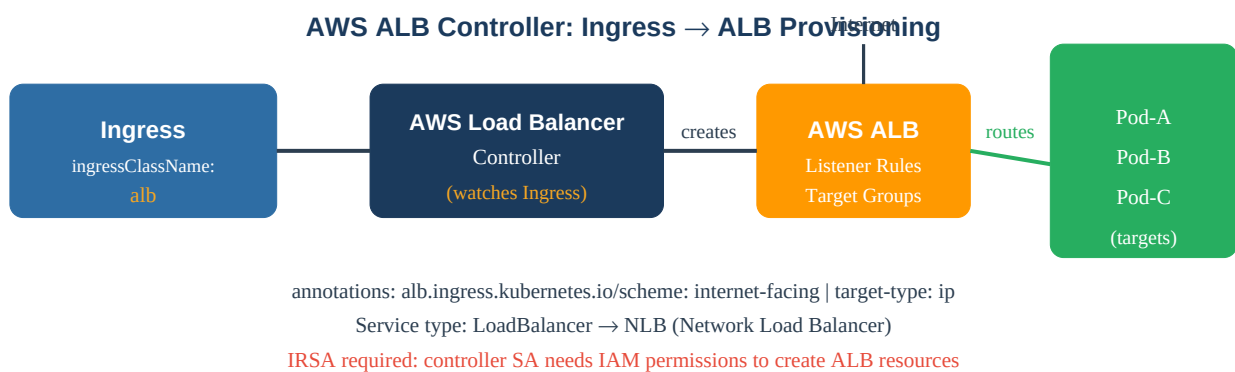
```
# 1. Create OIDC provider for cluster eksctl utils associate-iam-oidc-provider \ --cluster
my-cluster \ --approve # 2. Create IAM Role with trust policy cat <<EOF > trust-policy.json {
"Version": "2012-10-17", "Statement": [ { "Effect": "Allow", "Principal": { "Federated":
"arn:aws:iam::123456:oidc-provider/oidc.eks.us-east-1.amazonaws.com/id/XXXX" }, "Action":
"sts:AssumeRoleWithWebIdentity", "Condition": { "StringEquals": {
"oidc.eks.us-east-1.amazonaws.com/id/XXXX:sub": "system:serviceaccount:app-ns:app-sa",
"oidc.eks.us-east-1.amazonaws.com/id/XXXX:aud": "sts.amazonaws.com" } } ] } EOF aws iam
create-role --role-name app-role --assume-role-policy-document file://trust-policy.json aws
iam attach-role-policy --role-name app-role --policy-arn
arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess # 3. Annotate k8s ServiceAccount kubectl
annotate serviceaccount app-sa \ --namespace app-ns \
eks.amazonaws.com/role-arn=arn:aws:iam::123456:role/app-role
```

EKS Fargate Profiles

- **Serverless pods:** each pod runs on its own micro-VM (Firecracker) — no node management
- Match pods by namespace + label selectors in Fargate Profile
- No DaemonSets on Fargate, no GPU, no privileged containers
- Billing: per vCPU and memory per second
- Coexist with managed node groups — use Fargate for burstable, nodes for stable workloads
- Pod startup slower (~30s) vs node-based scheduling

```
# Fargate profile aws eks create-fargate-profile \ --cluster-name my-cluster \
--fargate-profile-name app-profile \ --pod-execution-role-arn
arn:aws:iam::123456:role/eks-fargate-pod-execution \ --selectors
namespace=app-ns,labels={tier=api} # eksctl equivalent eksctl create fargateprofile \
--cluster my-cluster \ --namespace app-ns \ --labels tier=api
```

AWS Load Balancer Controller



- **Replaces AWS in-tree Ingress controller:** manages ALB (Ingress) and NLB (Service LoadBalancer)
- **Target types:** instance (NodePort) or ip (direct pod IP — more efficient, requires VPC-CNI)
- **IngressGroup:** merge multiple Ingress resources into one ALB (cost saving)
- **IRSA required:** controller SA needs IAM permissions to create/manage ALB resources

```
# Key Ingress annotations for ALB
alb.ingress.kubernetes.io/scheme: internet-facing alb.ingress.kubernetes.io/target-type: ip
alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:us-east-1:123:certificate/xxx
alb.ingress.kubernetes.io/ssl-redirect: "443" alb.ingress.kubernetes.io/group.name: shared-alb
# IngressGroup alb.ingress.kubernetes.io/healthcheck-path: /health
alb.ingress.kubernetes.io/load-balancer-attributes: | deletion_protection.enabled=true #
Service as NLB annotations: service.beta.kubernetes.io/aws-load-balancer-type: external
service.beta.kubernetes.io/aws-load-balancer-nlb-target-type: ip
service.beta.kubernetes.io/aws-load-balancer-scheme: internet-facing
```

Cluster Upgrades

Strategy	Steps	Downtime	Rollback
In-place rolling	Upgrade CP, then node groups one by one	None (if PDB set)	Hard — no old cluster
Blue-green	New cluster, migrate workloads	Minimal (DNS cutover)	Easy — switch DNS back
Node group rotation	New node group, drain old	None (pod rescheduling)	Delete new node group

EKS Add-ons

- **EKS Add-ons:** AWS-managed lifecycle for core cluster components
- Available: vpc-cni, coredns, kube-proxy, aws-ebs-csi-driver, aws-efs-csi-driver
- AWS handles version updates, security patches, compatibility validation
- Conflicts: EKS add-on takes ownership — manual changes may be overwritten

Interview Q&A; — GKE & EKS Managed Kubernetes

Q1. What is the difference between GKE Standard and GKE Autopilot?

GKE Standard: you provision and manage node pools (VM type, size, OS), configure Cluster Autoscaler, patch node OS. Google manages control plane. Pay per node VM. GKE Autopilot: Google manages ALL infrastructure including nodes. You only deploy workloads. Pay per pod (vCPU + memory). Enforces resource requests on all pods. Autopilot has constraints: no DaemonSets, no SSH to nodes, no privileged containers, required resource requests. Use Autopilot for simplicity, Standard for advanced node control.

Q2. How does GKE Workload Identity work?

Workload Identity binds a Kubernetes ServiceAccount to a GCP Service Account. When a pod uses the annotated k8s SA, the GKE metadata server exchanges the k8s SA JWT for short-lived GCP access tokens. The pod's client libraries (GCS, BigQuery SDK) automatically detect and use these credentials via the metadata server. No key file management needed. Annotate k8s SA: `iam.gke.io/gcp-service-account=gsa@project.iam`.

Q3. What is EKS IRSA and how does it differ from kiam/kube2iam?

IRSA (IAM Roles for Service Accounts) uses OIDC token exchange. Pod gets projected JWT with audience `sts.amazonaws.com`. AWS SDK calls `AssumeRoleWithWebIdentity` with this token. STS validates JWT against EKS OIDC provider, returns temporary credentials. kiam/kube2iam: run as DaemonSet, intercept EC2 metadata calls, proxy to STS. IRSA is preferred: built into AWS SDK, no additional components, pod-level isolation, works with Fargate, no metadata server interception needed.

Q4. How do you configure Fargate profiles in EKS?

Fargate profiles define which pods run on Fargate using namespace + label selectors. If a pod matches a profile, it's scheduled onto Fargate (micro-VM). If no profile matches, it's scheduled onto EC2 nodes. Fargate profile requires pod execution IAM role (for pulling images, CloudWatch logs). Limitations: no DaemonSets, no GPU, no host networking, no privileged containers, no local ephemeral storage beyond `emptyDir`. Startup time ~30s vs sub-10s for nodes.

Q5. What is the AWS Load Balancer Controller and what does it replace?

AWS Load Balancer Controller replaces the deprecated in-tree AWS load balancer support. For Ingress resources: creates and manages Application Load Balancers (ALB) with path/host-based routing. For Service type: `LoadBalancer`: creates Network Load Balancers (NLB). Key improvements: target-type: ip routes directly to pod IPs (bypasses NodePort), IngressGroup merges multiple Ingresses into one ALB, better annotation-based configuration. Requires IRSA with `elasticloadbalancing` and `ec2` IAM permissions.

Q6. What is the EKS VPC CNI plugin?

vpc-cni (aws-node) assigns AWS VPC IPs directly to pods — each pod gets a real ENI secondary IP. Pods are fully routable within VPC — no overlay network (no VXLAN). Benefits: direct VPC routing, security group per pod (SGPP feature), native AWS traffic inspection. Limitation: IP exhaustion risk (each node has limited ENIs/IPs). Mitigation: CIDR prefix delegation, IPv6 clusters, or use Karpenter for right-sized nodes.

Q7. How do you upgrade an EKS cluster?

1) Upgrade control plane: `aws eks update-cluster-version` (or `eksctl`). 2) Update EKS add-ons (vpc-cni, coredns, kube-proxy) to versions compatible with new k8s. 3) Upgrade managed node groups: `eksctl upgrade nodegroup` (launches new nodes, drains old). 4) Test workloads on new version before upgrading node groups. AWS supports n-2 minor versions. In-place upgrades are supported but blue-green gives easier rollback.

Q8. What is Binary Authorization in GKE?

Binary Authorization is a deploy-time security control that requires container images to be cryptographically signed before they can be deployed. Policy: only images attested by specific signers (build system, security scanner) are allowed. Attestors verify signatures stored in Container Analysis. If an unsigned image is deployed, Binary Authorization webhook rejects the pod. Use case: enforce that only images that passed CI/CD and vulnerability scanning can run in prod.

Q9. How does GKE Cloud Armor protect Kubernetes workloads?

Cloud Armor is GCP's WAF and DDoS protection service, attached to GCP Cloud Load Balancer. GKE Ingress (with BackendConfig CRD) attaches Cloud Armor security policy to the backend service. Features: OWASP top-10 protection, IP allowlist/blocklist, rate limiting, geo-based rules. Traffic: Internet -> Cloud Armor -> GCP LB -> GKE Ingress -> Pod. Config: BackendConfig.spec.securityPolicy.name references the Cloud Armor policy.

Q10. What are EKS managed node groups vs self-managed node groups?

Managed node groups: AWS automates node provisioning (EC2 ASG), OS patching (EKS-optimized AMI updates), node drain during updates. You choose instance type, size, capacity. Self-managed nodes: you manage EC2 instances, AMI updates, autoscaling configuration, drain logic. Full control of OS, instance configuration, custom AMIs. Karpenter replaces both with just-in-time node provisioning optimized per pending pod requirements.

Q11. What is GKE Dataplane V2?

GKE Dataplane V2 uses eBPF (via Cilium) instead of iptables/kube-proxy for packet processing. Benefits: better performance (fewer kernel hops), built-in NetworkPolicy enforcement (no separate CNI), network observability (Hubble), L7 network policies. kube-proxy not needed — eBPF handles service load balancing in kernel directly. Enable on cluster creation: --dataplane-v2 flag.

Q12. How does EKS handle etcd?

EKS fully manages etcd as part of the managed control plane. etcd runs in multiple AZs (HA). AWS handles backups, encryption (KMS), version upgrades. You cannot access etcd directly — only through k8s API server. etcd backup/restore not needed by customers (AWS manages). Cost: \$0.10/hour per EKS cluster (covers control plane + etcd HA).

Q13. What is Karpenter and how does it differ from Cluster Autoscaler on EKS?

Karpenter: open-source node provisioner (built by AWS). Directly calls EC2 APIs to provision optimal instance types matching pending pod requirements. No pre-defined node groups needed. Provisioners define constraints (instance families, AZs, spot/on-demand). Cluster Autoscaler: scales pre-defined ASG node groups. Slower, less flexible. Karpenter advantages: 60s vs 3-4min for node availability, consolidation (replace underused nodes), instance diversity (picks best instance type for cost/performance).

Q14. How do you implement pod identity in EKS without IRSA?

EKS Pod Identity (launched 2023): newer alternative to IRSA. Maps k8s SA to IAM role via EKS Pod Identity Association (API/console). No need to configure OIDC provider trust policy. Agent DaemonSet (eks-pod-identity-agent) intercepts credential requests and exchanges for AWS credentials. Simpler setup than IRSA but requires eks-pod-identity-agent add-on. IRSA: still supported and widely used; works without agent. Both are superior to node IAM roles.

Q15. What is EKS Blueprints?

EKS Blueprints: opinionated Terraform (and CDK) modules for production-ready EKS cluster setup. Includes: EKS cluster, managed node groups, Fargate profiles, core add-ons (ALB controller, Cluster Autoscaler/Karpenter, ArgoCD, metrics-server, external-dns, cert-manager). GitOps-ready: ArgoCD

bootstrapped to deploy application add-ons. Reduces EKS setup from weeks to hours. Community-maintained best practices.

Q16. How do you handle multi-tenancy on EKS?

1) Namespace isolation: separate namespaces per team with NetworkPolicies and RBAC. 2) Separate clusters: stronger isolation, higher cost. 3) EKS Node Selectors: dedicated node groups per tenant (taint nodes, use tolerations). 4) IRSA per team: separate IAM roles per k8s SA. 5) Resource Quotas: limit CPU/memory/PVC per namespace. 6) Kyverno/OPA: enforce naming conventions, image registry allowlists per namespace. For sensitive compliance (PCI, HIPAA): separate clusters recommended.

Q17. What is the GKE Config Connector?

Config Connector is a GKE add-on that allows managing GCP resources (CloudSQL, Pub/Sub topics, GCS buckets, BigQuery datasets) as Kubernetes CRDs. Create a SQLInstance CRD in k8s → Config Connector provisions a Cloud SQL instance in GCP. GitOps-friendly: GCP infrastructure defined as YAML in Git, managed by ArgoCD/Flux. Requires Workload Identity for controller SA to manage GCP resources.

Q18. How does GKE node auto-provisioning work?

Node Auto-Provisioning (NAP) automatically creates and deletes node pools based on pending pod requirements. Unlike Cluster Autoscaler (scales existing pools), NAP creates new pools with optimal machine types. Configuration: enable via `--enable-autoprovisioning`, set resource limits (max CPU/memory per pool). Uses Cluster Autoscaler under the hood. GKE Autopilot is the managed version — no configuration needed.

Q19. What are EKS security groups for pods?

Security Groups for Pods (SGPP) allows attaching specific EC2 security groups to pods (not just nodes). Use case: pod needs access to RDS database — attach the DB security group to the pod directly, no node-level firewall change. Requirements: VPC CNI SGPP mode, instance types supporting multiple ENI (no t2/t3 micro). Pod gets ENI with assigned security group — VPC-level traffic filtering.

Q20. How do you implement cluster upgrade for zero downtime on GKE?

GKE control plane upgrade: instantaneous, zero downtime (GKE manages HA). Node pool upgrade: GKE creates new nodes (with new k8s version), cordons old nodes, drains pods (respects PDB), terminates old nodes. Requirements for zero downtime: PodDisruptionBudgets set, multiple replicas, readiness probes, anti-affinity to spread across nodes. Surge upgrades: add surge nodes before draining (no capacity shortage). `maxSurge` and `maxUnavailable` configurable per node pool.

Q21. What is the AWS VPC CNI prefix delegation mode?

Default ENI mode: each ENI secondary IP address = 1 pod IP. Limited by ENI count and IPs per ENI. Prefix delegation: each ENI gets a /28 CIDR prefix (16 IPs) instead of individual IPs. Dramatically increases pod density per node. Requires VPC subnet with adequate CIDR space. Enable: `ENABLE_PREFIX_DELEGATION=true` on `vpc-cni`. Essential for large clusters where default IP limits cause 'too many pods' node capacity errors.

Q22. What observability does GKE provide out of the box?

Cloud Logging: container stdout/stderr automatically collected (no Fluentd config needed). Cloud Monitoring: cluster, node, pod metrics auto-populated in GKE dashboards. Managed Prometheus: Google Cloud Managed Service for Prometheus — no Prometheus ops. GKE Dataplane V2 + Hubble: eBPF-based network observability (flows, policy verdicts). Audit logs: GKE API server audit logs sent to Cloud Audit Logs. Binary Authorization audit events in Cloud Audit Logs.

Q23. How does EKS integrate with AWS CloudWatch for monitoring?

CloudWatch Container Insights: deploys CloudWatch agent as DaemonSet, collects cluster/node/pod metrics. CloudWatch Logs: Fluent Bit DaemonSet (aws-for-fluent-bit) ships container logs. EKS control plane logging: enable API, audit, authenticator, controllerManager, scheduler logs via AWS Console/CLI — sent to CloudWatch Log Groups. Amazon Managed Prometheus + Managed Grafana: for Prometheus-compatible metrics without ops overhead.

Q24. What is the EKS pod identity association vs IRSA trust policy?

IRSA trust policy: manually create IAM role with trust policy containing OIDC condition, then annotate k8s SA with role ARN. Complex setup, must match namespace/SA name in condition. EKS Pod Identity Association: create mapping via EKS API (no manual trust policy editing). eks-pod-identity-agent DaemonSet handles credential vending. Simpler: `aws eks create-pod-identity-association --cluster my-cluster --namespace app-ns --service-account app-sa --role-arn arn:aws:iam::123:role/app-role`.

Q25. How do you handle node OS patching in EKS?

Managed node groups: AWS releases new EKS-optimized AMIs. Trigger update via `eksctl upgrade nodegroup --release-version` (rolling node replacement). Custom AMIs: Packer-based pipeline builds golden AMI with custom packages/agents, reference in launch template. Automated patching: Karpenter drift detection replaces nodes when AMI version changes. Bottlerocket OS: immutable container-focused OS from AWS — minimal attack surface, atomic updates.

Q26. What is GKE Fleet and what problems does it solve?

GKE Fleet (formerly Environ) manages multiple GKE clusters as a logical group. Provides: centralized Config Management (ACM — policy/config sync across clusters), Service Mesh across clusters (Traffic Director, managed Anthos Service Mesh), Multi-cluster Ingress (single Ingress routing across regional clusters), fleet-wide compliance policies. Use for: multi-region deployments, dev/staging/prod cluster governance, hybrid cloud (GKE + Anthos on-prem).